

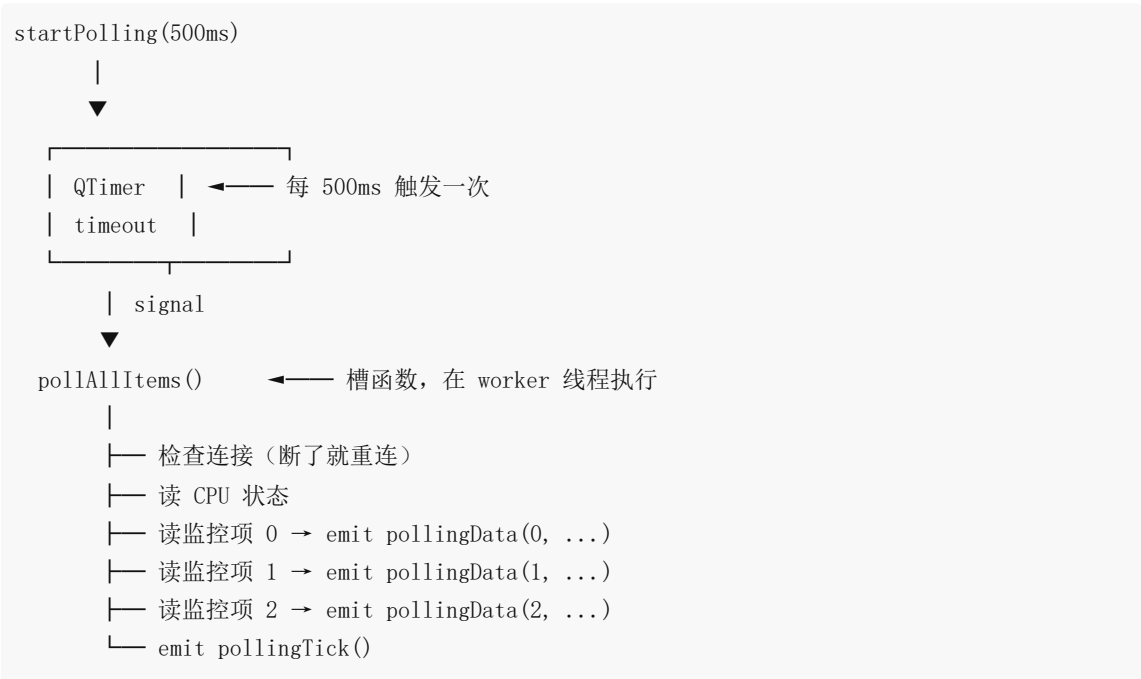
Qt + Snap7 实时监控 PLC —— 小白上手指南（项目 5）

本项目和项目 4 的区别

	项目 4	项目 5
数据刷新	手动点"读取"	自动周期刷新 (QTimer)
地址管理	单次读写一个地址	地址表——一次配多个，同时监控
新增控件	—	监控地址表 (QWidget) 、轮询开关
连接维护	—	断线自动重连
CPU 状态	—	实时显示 RUN / STOP
核心新知识	—	QTimer、QWidget、状态机思维

QTimer 的工作原理

QTimer 是 Qt 的定时器——设定一个时间间隔，到点就触发一次回调。



QTimer 放在哪个线程很重要： 项目中 QTimer 是 PlcWorker 的成员，PlcWorker 通过 `moveToThread` 移到了子线程。所以 `pollAllItems()` 在子线程执行，阻塞的 snap7 读操作不会卡 UI。

项目文件一览

```
stage5_polling/
├── CMakeLists.txt
```

—— main.cpp	← 同前
—— mainwindow.h	← 新增监控表、轮询控件声明
—— mainwindow.cpp	← 新增 QTableWidgetItem 管理、轮询启停
—— plc_worker.h	← 新增 MonitorItem 结构体、QTimer、轮询接口
—— plc_worker.cpp	← 新增 pollAllItems、自动重连、CPU 状态
—— snap7.h / snap7.cpp / snap7.dll / libsnap7_mingw.a	

新增数据结构：MonitorItem

```
struct MonitorItem {
    int area    = AREA_M;    // 区域：M=0, I=1, Q=2, DB=3
    int dbNum   = 0;         // DB 号（仅 DB 区使用）
    int start   = 0;         // 字节起始地址
    int valType = 4;         // 解析类型：4=UINT16
};
```

一个 MonitorItem 描述一个“要监控的地址”的完整信息。多个 MonitorItem 组成一个列表，轮询时逐个读取。

数据流：

```
用户添加表行 → QTableWidgetItem 每行存一套控件
|
▼ onStartPolling()
syncItemsToWorker() → 遍历 QTableWidgetItem → 提取 MonitorItem 列表
|
▼ emit requestSetMonitorItems(items)
PlcWorker::setMonitorItems() → 存到 m_items
|
▼ (每 500ms)
pollAllItems() → 逐项读取 → emit pollingData(idx, data)
|
▼
onPollingData() → 根据 idx 找到对应行 → 更新“当前值”列
```

关键代码讲解

QTimer 的创建与绑定 (plc_worker.cpp 构造函数)

```
PlcWorker::PlcWorker(QObject *parent) : QObject(parent)
{
    m_client = new TS7Client();
    m_timer  = new QTimer(this);    // QTimer 属于 worker，在 worker 线程运行

    // 把 QTimer 的 timeout 信号连到 pollAllItems
```

```
connect(m_timer, &QTimer::timeout, this, &PlcWorker::pollAllItems);  
}
```

关键：QTimer 用 `this` 做 parent，而 `this` (PlcWorker) 在 worker 线程。所以 timeout 信号在 worker 线程触发，`pollAllItems` 也在 worker 线程执行。

启动与停止轮询

```
void PlcWorker::startPolling(int intervalMs)  
{  
    m_timer->start(intervalMs);    // start(0) = 尽可能快地重复触发  
}  
  
void PlcWorker::stopPolling()  
{  
    m_timer->stop();  
}
```

轮询主循环

```
void PlcWorker::pollAllItems()  
{  
    // ---- 1. 自动重连 ----  
    if (!m_client->Connected() && !m_lastIp.isEmpty()) {  
        // 用上次成功连接的参数重连  
        m_client->ConnectTo(m_lastIp.toUtf8().constData(),  
                           m_lastRack, m_lastSlot);  
    }  
  
    if (!m_client->Connected()) {  
        emit disconnected();    // 重连失败，通知 UI  
        return;  
    }  
  
    // ---- 2. 读 CPU 状态 ----  
    int status = m_client->PlcStatus();  
    emit cpuStatusChanged(status);  
  
    // ---- 3. 逐项读取 ----  
    for (int i = 0; i < m_items.size(); ++i) {  
        const MonitorItem &item = m_items[i];  
        // ... 根据 item.area 调用对应 snap7 读函数 ...  
        emit pollingData(i, item.area, item.dbNum, item.start, buffer);  
    }  
}
```

自动重连机制

连接断开有两种场景：

1. 网络断线、PLC 断电 → `Connected()` 返回 `false`
2. 首次连接失败 → `m_lastIp` 为空，不尝试重连

每次轮询周期开始时检查，如果断了就尝试用之前保存的 IP/Rack/Slot 重连。重连成功则继续轮询，失败则 `emit disconnected()`。

QTableWidget 嵌入控件

```
void MainWindow::onAddMonitorItem()
{
    int row = m_table->rowCount();
    m_table->insertRow(row);

    // 给每列嵌入不同的控件
    QComboBox *areaCb = new QComboBox();
    areaCb->addItem({"M", "I", "Q", "DB"});
    m_table->setCellWidget(row, 0, areaCb);

    QSpinBox *startSpin = new QSpinBox();
    startSpin->setRange(0, 65535);
    m_table->setCellWidget(row, 2, startSpin);

    // ... 其他列类似 ...
}
```

`setCellWidget(row, col, widget)` 是 `QTableWidget` 的核心方法——把任意 Qt 控件嵌到表格单元格里。读取时用 `cellWidget(row, col)` 取出来，`qobject_cast` 转回原类型。

syncItemsToWorker() —— 从表中提取数据

```
void MainWindow::syncItemsToWorker()
{
    QVector<MonitorItem> items;
    for (int i = 0; i < m_table->rowCount(); ++i) {
        auto *areaCb = qobject_cast<QComboBox*>(m_table->cellWidget(i, 0));
        auto *typeCb = qobject_cast<QComboBox*>(m_table->cellWidget(i, 3));
        // ... 从每个控件取当前值 ...
        MonitorItem item;
        item.area = areaCb->currentIndex();
        item.start = startSpin->value();
        item.valType = typeCb->currentData().toInt();
        items.append(item);
    }
}
```

```
emit requestSetMonitorItems(items);  
}
```

CPU 状态解读

返回值	含义	界面显示
0x08	RUN — PLC 正在执行程序	绿色 "CPU: RUN"
0x04	STOP — PLC 已停止	红色 "CPU: STOP"
其他	未知状态	橙色显示十六进制值

轮询时间间隔的选择

间隔	场景	注意事项
100ms	高速监控（少量地址）	每条读操作约 5-20ms，10 个地址 ≈ 50-200ms，100ms 可能来不及
500ms	常规监控（推荐）	10 个地址轻松跑完
2000ms	慢速监控、日志记录	网络开销小

如果轮询间隔小于实际读取耗时，下次 timeout 会排队，不会丢失触发。但值会逐渐落后于实际。

UI 布局

连接

PLC IP: [192.168.0.1] [连接] [断开]

● 已连接CPU: RUN ●

监控地址表（轮询）

区域	DB	起始	类型	当前值	状态
M	—	0	U16	100	OK
DB	1	4	F32	3.14	OK
Q	—	0	BOL	1 0 1...	OK

[+添加] [-删除]

轮询间隔: [500] ms [开始轮询] [停止]

手动读写（同项目4）

区域/起始/类型/值 [读取] [写入]

| HEX + 解析值 + 写入结果 |

编译步骤

```
cmake -B build -G "Ninja" \
    -DCMAKE_CXX_COMPILER=g++ \
    -DCMAKE_PREFIX_PATH=C:/Qt/6.11.0/mingw_64
cmake --build build
cp snap7.dll build/
export PATH="/c/Qt/6.11.0/mingw_64/bin:$PATH"
cd build && windeployqt Stage5_Polling.exe
```

常见问题

Q: 点"开始轮询"后按钮灰了但值不刷新

原因: 可能 `syncItemsToWorker()` 没有正确发送监控列表。 **解决:** 检查表中是否有行，每行的控件是否都存在。

Q: 某个地址显示"失败"

原因: 该地址超出 PLC 范围 / DB 未取消优化访问 / 类型选的字节数超过 DB 块大小。 **解决:** 单独用手动读写测试这个地址，确认能读到。

Q: 断开后不会自动重连

原因: `m_lastIp` 为空（从未连接成功过）。 **解决:** 至少成功连接过一次后，自动重连才会生效。

Q: 轮询期间 PLC 断电再上电，连不上了

原因: PLC 重启可能改变了某些连接参数。 **解决:** 先点"停止轮询"→"断开"→"连接"重新建立连接。

Q: CPU 状态一直显示 0x00

原因: `PlcStatus()` 在某些固件版本上返回值不标准。 **解决:** 试试通过读取 SZL（系统状态列表）获取更准确的状态。

五个项目知识体系回顾

项目	核心技能	snap7 函数
1 控制台	CMake + 编译链 + TS7Client 生命周期	ConnectTo / Disconnect
2 Qt 连接	Signal/Slot + QThread + moveToThread	同上
3 读取	大端解析 + PLC 存储区 + 类型转换	DBRead / MBRead / EBRead / ABRead

4 写入	大端打包 + 写后回读	DBWrite / MBWrite / EBWrite / ABWrite
5 轮询	QTimer + QTableWidgetItem + 自动重连 + CPU 状态	以上全部 + PlcStatus

从控制台一行 `ConnectTo`，到带图形界面的实时多地址监控系统——你现在已经能独立开发一个完整的 PLC 上位机了。

术语速查

术语	一句话解释
QTimer	Qt 定时器，按设定间隔触发 timeout 信号
QTableWidgetItem	表格控件，可以在单元格里嵌入任意 Qt 控件
setCellWidget	把控件放入表格单元格
qobject_cast	Qt 的安全类型转换（类似 C++ 的 dynamic_cast）
自动重连	检测到断线后，用之前保存的参数重新连接
pollingTick	自定义信号，标记一轮轮询完成
PlcStatus	snap7 方法，返回 CPU 的 RUN/STOP 状态